



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

INGENIERÍA EN COMPUTACIÓN

Laboratorio de Organización de Arquitectura de Computadoras

Reporte - Introducción a las herramientas de desarrollo de los FPGAs

NOMBRE DE LOS INTEGRANTES:

Omar Alfonso Zendejas Labias Grupo de Teoría: 04

NOMBRE DEL PROFESOR: ING. KAREN PÉREZ

GRUPO: 06

SEMESTRE: 2027-1.

FECHA DE ENTREGA: 3 de septiembre de 2026

Objetivo

En esta práctica se buscó que el alumno se familiarizara con el entorno de desarrollo Quartus Prime Lite Edition y con el flujo básico de diseño de sistemas digitales sobre una tarjeta de desarrollo basada en *FPGA*. A partir de la creación y configuración de un proyecto, la captura de un diagrama esquemático, su compilación, la asignación de señales de entrada y salida, y la realización de una simulación funcional mediante el *Simulation Waveform Editor*, se buscó que el alumno adquiriera la capacidad de llevar un diseño digital desde su concepción hasta su programación en una tarjeta *FPGA*.

Introducción

Un *Field Programmable Gate Array* (*FPGA*) es un dispositivo digital reconfigurable compuesto por una matriz de bloques lógicos y recursos de interconexión programables, que permiten implementar prácticamente cualquier circuito digital combinacional o secuencial sin necesidad de fabricar un circuito integrado dedicado. Para diseñar sobre estos dispositivos se emplean herramientas de diseño asistido por computadora (*Computer-Aided Design*, *CAD*); en el caso de los dispositivos de la familia MAX 10 utilizados en este laboratorio, la herramienta correspondiente es Quartus Prime, en su edición Lite.

Quartus Prime permite capturar un diseño de dos formas principales: mediante lenguajes de descripción de hardware (como VHDL o Verilog) o mediante *schematic entry*, es decir, dibujando directamente el circuito con símbolos de compuertas lógicas interconectadas. Esta segunda modalidad es la que se utilizó en la presente práctica, en la cual el circuito se construyó a partir de compuertas XOR, NAND, OR, NOR y AND, cuyo comportamiento se rige por las reglas del álgebra booleana descritas, entre otros, por Mano y Ciletti [1]. Una vez capturado y compilado el diseño, su comportamiento lógico puede verificarse antes de programarlo físicamente mediante una simulación funcional, en la que se generan formas de onda para las entradas y se observa la respuesta calculada para las salidas a través del *Simulation Waveform Editor*.

Desarrollo

Creación del proyecto

Se creó un nuevo proyecto en Quartus Prime Lite Edition mediante el asistente *New Project Wizard*, seleccionando como familia de dispositivo MAX 10 (DA/DF/DC/SA/SC), tal como lo indica

el manual de la práctica. Dado que para esta sesión no se contó con la tarjeta de desarrollo física, se conservó el dispositivo asignado por defecto dentro de dicha familia en lugar de seleccionar manualmente un modelo específico. Como herramienta de simulación se configuró ModelSim-Altera con formato VHDL, de acuerdo con lo especificado en el manual.

Diseño esquemático

Una vez creado el proyecto, se agregó un archivo de tipo *Block Diagram/Schematic File* y se construyó el circuito combinacional solicitado, con tres entradas (A, B y C) y una salida (S), utilizando compuertas XOR, NAND, OR, NOR y AND interconectadas según se muestra en la figura 1.

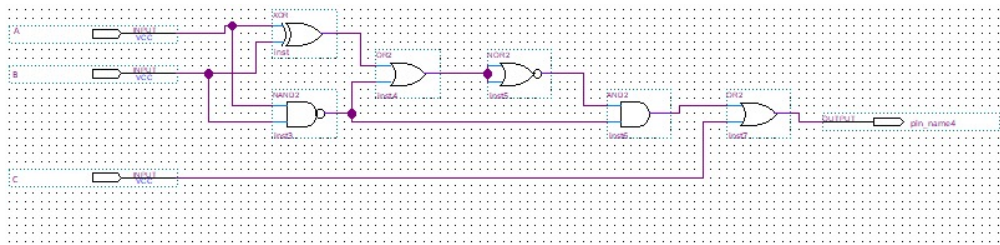


Figura 1: Diagrama esquemático del circuito combinacional implementado en Quartus Prime, con entradas A, B, C y salida S.

Las entradas A y B alimentan una compuerta XOR y una compuerta NAND. Las salidas de estas dos compuertas se combinan mediante una compuerta OR y, posteriormente, mediante una compuerta NOR, para finalmente combinarse de nuevo con la salida de la compuerta NAND en una compuerta AND. El resultado de esta red de cinco compuertas se combina, por último, con la entrada C mediante la compuerta OR final (renombrada con el apellido del alumno, conforme lo pide el manual) para producir la salida S.

Analizando la red mediante álgebra booleana puede verificarse que, para las entradas A y B, la salida de la compuerta OR (que combina el XOR y el NAND de A y B) es equivalente al propio NAND de A y B, ya que el XOR nunca vale 1 cuando el NAND vale 0. La compuerta NOR posterior invierte ese resultado, entregando el AND de A y B; y la compuerta AND final de esta subred combina dicho valor con el NAND de A y B, que es su complemento, por lo que el resultado es siempre 0 independientemente de A y B:

$$\text{or}_1 = (A \oplus B) + \overline{AB} = \overline{AB} \quad (1)$$

$$\text{nor}_1 = \overline{\text{or}_1} = \overline{\overline{AB}} = AB \quad (2)$$

$$\text{and}_1 = \text{nor}_1 \cdot \overline{AB} = AB \cdot \overline{AB} = 0 \quad (3)$$

Como la compuerta OR final combina este valor constante (ecuación 3) con la entrada C, la salida del circuito se reduce a $S = 0 + C = C$ para cualquier combinación de A y B. Esta reducción algebraica es consistente con el comportamiento observado durante la simulación funcional, descrita en la sección siguiente.

Simulación funcional

Para verificar el comportamiento del circuito se creó un archivo de tipo *University Program VWF* mediante el *Simulation Waveform Editor*. Las señales A, B, C y S se insertaron utilizando la herramienta *Node Finder*, copiando todos los nodos encontrados a la lista de nodos seleccionados. Posteriormente se generaron, sobre las entradas A, B y C, las ocho combinaciones posibles (000, 001, 010, 011, 100, 101, 110 y 111) solicitadas en el manual, y se ejecutó la simulación funcional (*Run Functional Simulation*).

3.3.1. Tabla de verdad

A partir del comportamiento observado en la simulación, la tabla de verdad del circuito es la siguiente:

A	B	C	S
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

Tabla 1: Tabla de verdad del circuito combinacional.

Como se aprecia en la tabla 1, la salida S coincide en todos los casos con el valor de la entrada C,

sin depender de A ni de B, lo cual concuerda con la reducción algebraica presentada en la ecuación 3.

Análisis de resultados

Durante la primera ejecución de la simulación se obtuvo el resultado mostrado en la figura 2, en el cual la salida S permanece en estado desconocido (X) durante toda la simulación, a pesar de que las entradas A, B y C sí presentan las transiciones configuradas.

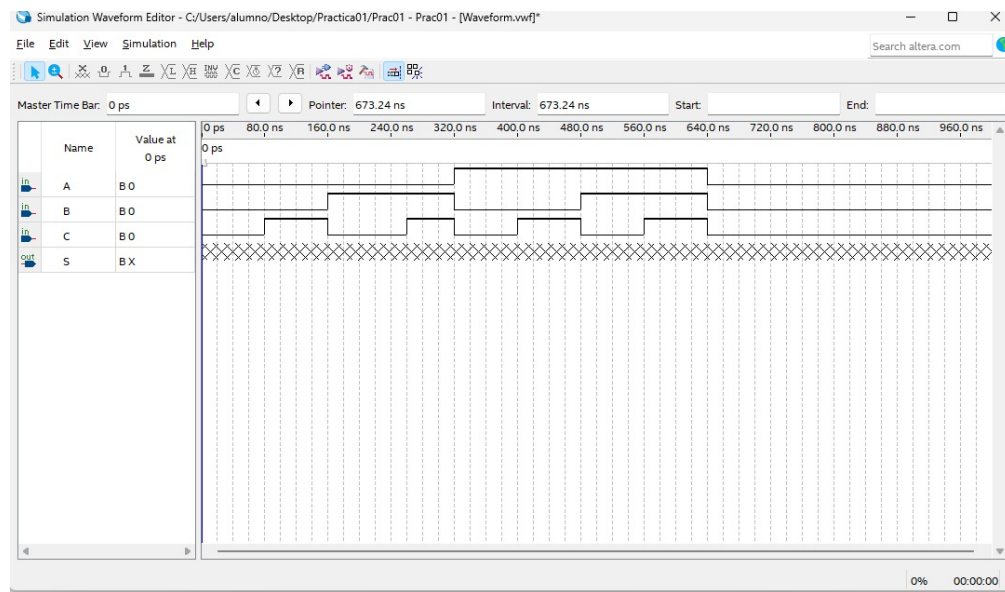


Figura 2: Simulación funcional con salida en estado desconocido (X), evidencia de una conexión incorrecta en el diseño.

Este resultado corresponde al fallo lógico ante el cual advierte el manual de la práctica. En este caso el problema no estuvo en el cableado del esquemático, sino en la ubicación del proyecto: éste se había creado directamente en la raíz del equipo, lo que generaba conflictos y no dejaba que la simulación resolviera un valor lógico definido para S. La corrección consistió en cerrar Quartus Prime por completo y volver a crear el proyecto, con el mismo diagrama, dentro de una carpeta dedicada en lugar de la raíz.

Tras rehacer el proyecto de esta manera, se volvió a ejecutar la simulación funcional, obteniéndose el resultado mostrado en la figura 3.

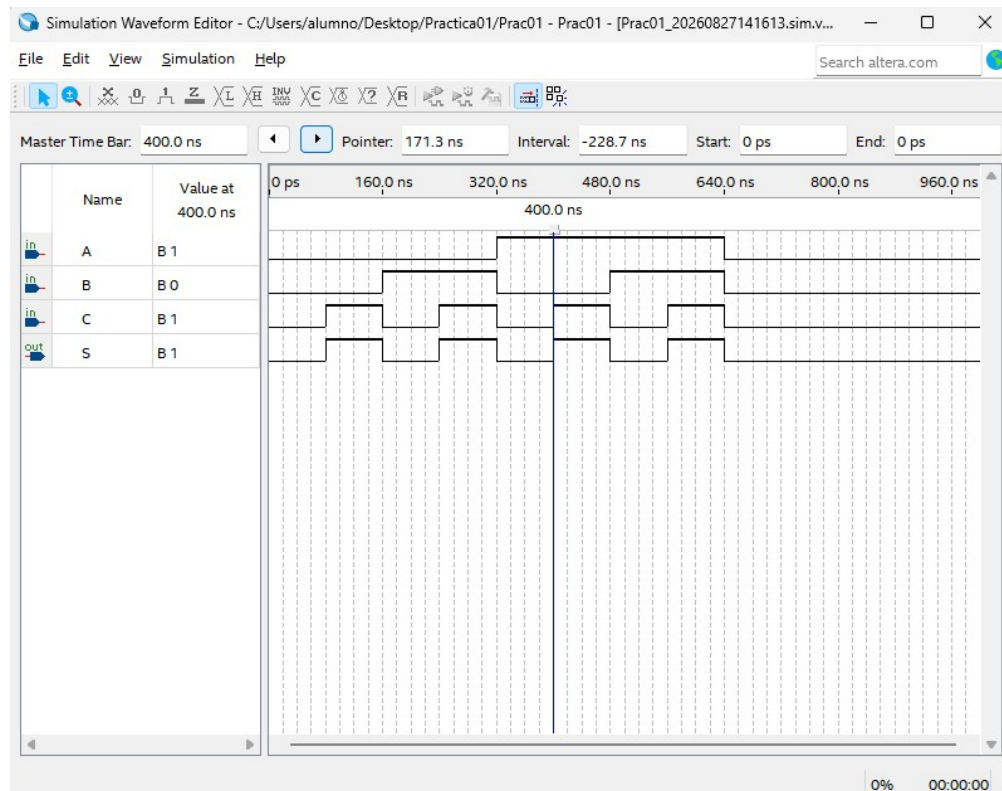


Figura 3: Simulación funcional correcta mostrando las ocho combinaciones de entrada y el valor resultante de la salida S.

En este segundo resultado, la salida S sí toma valores lógicos definidos y, para cada una de las ocho combinaciones de entrada generadas, reproduce el valor de la entrada C, tal como se anticipó en la tabla 1. Por ejemplo, en el instante marcado por la barra de tiempo (400.0 ns), con $A = 1$, $B = 0$ y $C = 1$, la salida observada es $S = 1$, consistente con $S = C$.

La etapa de asignación de pines y programación de la tarjeta FPGA no se abordó en esta sesión, ya que aún no se trabajó con la tarjeta física; dicha etapa se cubrirá en una práctica posterior.

Conclusiones individuales

Omar Alfonso Zendejas Labias

El objetivo de la práctica se cumplió: logré crear el proyecto en Quartus Prime, capturar el diagrama esquemático con las compuertas indicadas y simular su funcionamiento con el Simulation Waveform Editor. El único inconveniente fue que la primera simulación arrojaba la salida S en estado desconocido (X); esto no se debía al circuito, sino a que el proyecto se había creado en la raíz del equipo, así que lo rehice dentro de una carpeta y con eso la simulación arrojó valores definidos.

Como resultado, la tabla de verdad confirma que la salida S siempre es igual a la entrada C, sin depender de A ni de B. Queda pendiente la etapa de asignación de pines y programación en la tarjeta física, que aún no se ha trabajado en el curso.

Referencias

- [1] M. M. Mano y M. D. Ciletti, *Diseño digital*, 5a ed. Naucalpan de Juárez, México: Pearson Educación, 2013.